

40. Quatre nuances, quatre signes

Une tuile s'écrit en huit rangées de huit caractères.

1. Ce qu'il faut comprendre

`.` clair, `-` moyen clair, `+` moyen sombre, `#` sombre. On peut aussi écrire `0123` — c'est la même chose.

Le nom `BRIQUE` devient un **numéro de tuile** : il s'écrit partout où l'on attend un numéro. Le compilateur grave les seize octets dans la cartouche et range la tuile à sa place au démarrage.

Dans la page, cette même tuile se dessine **à la souris** — l'atelier écrit exactement ces huit lignes dans le programme. Les deux chemins mènent au même endroit.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Tuile>` → [0.107.2. Tuile — dessiner une case](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **210 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage - la mise en route de la console
main()         38 o ← démarrage - ta fonction
VBlank         8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage  28 o ← image() - attend l'image suivante
AttendreVBlank 16 o ← démarrage - attend que l'écran soit libre
AdresseCase    10 o ← poser() - trouve l'adresse d'une case de l'écran
CopierTuiles   31 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage - vide les deux cartes de l'écran
Tuiles         16 o ← CopierTuiles - 0 lettre sur 43, et 1 tuile dessinée
```

Lettres gravées : aucune

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreAcces`, `AttendreFinVBlank`, `EffacerCases`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

```
poser() × 1    13 o sur place, dans main() + AdresseCase (10 o)
image() × 1    3 o sur place, dans main() + AttendreImage (28 o)
```

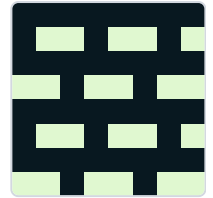
1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : dessine sa propre tuile, une BRIQUE, et en pose une rangée.
//
// Une tuile : 8 rangées de 8 pixels. Chaque signe est un pixel, en 4 nuances :
// . clair      - moyen clair    + moyen sombre    # sombre
// (on peut aussi écrire 0 1 2 3 : c'est la même chose)
// Le nom BRIQUE devient un NUMÉRO de tuile, utilisable avec poser().

#include <Tuile>    // un dessin de 8 × 8 pixels
#include <poser>    // pose une tuile sur une case du fond
```

2. UN DESSIN — BRIQUE

```
Tuile BRIQUE = {  
    "#####",  
    "#..#..#.",  
    "#####",  
    "..#..#..",  
    "#####",  
    "#..#..#.",  
    "#####",  
    "..#..#..",  
};
```



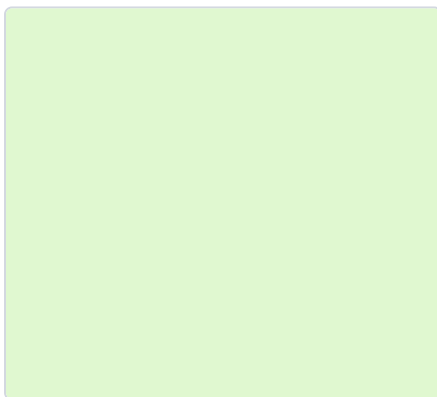
à l'écran

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

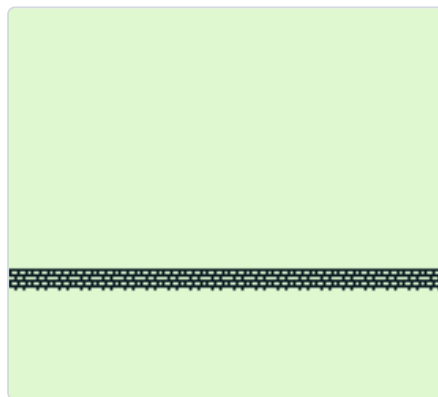
```
int main() {                                     // Le jeu commence ici.  
    for (uint8_t x = 0; x < 20; x++) poser(x, 12, BRIQUE);  
                                                // for : x va de 0 à 19 (20 tours).  
                                                // À chaque tour : une brique en  
                                                // colonne x, ligne 12. Toute la ligne.  
  
    while (true) {                               // Répète pour toujours :  
        image();                                 // attend l'image suivante.  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Une rangée de briques en travers de l'écran.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ la rangée est posée d'un bord à l'autre
- ✓ ce n'est pas une lettre de la police (n° 44)
- ✓ le reste de l'écran est vide

6. La même chose, à la souris

Cette leçon porte son atelier : le geste réécrit le programme ci-dessus, et la cartouche se refait dans la seconde. C'est le même texte, écrit autrement.

urs et fonctions, une seule suite pour écrire une cartouche Game Boy en C++

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

chercher un dessin : nom, étiquette, ★

BRIQUE

+ tuile 8 × 8

+ perso 16 × 16

Game Boy — 4 nuancesLes quatre nuances de la console d'origine, et rien de plus.

Passer en couleur

nuance 3

Nuances

BRIQUE

tuile n° 44 — poser(x, y, BRIQUE)

– zoom

22×

+ zoom

Plein écran

Peindre ✓

Gomme

/ Ligne

Rectangle

Cercle

Formes : contour

Remplir

Pipette

Sélectionner et déplacer

Miroir

Miroir

Tourner

Copier le dessin

Coller ici

Dupliquer

Effacer la tuile

taille réelle — 8 × 8 pixels

étiquettes décor mur...

Animation

BRIQUE ne bouge pas. Une animation toute prête (elle fabrique les images) :

Eau (vague)

Feu (flamme)

Herbe (vent)

Clignoter

+ À la main

Police

BRIQUE peut remplacer une lettre de la police : tous les textes du jeu la montreront avec ce dessin.

remplacer la lettre...

Ensuite : 41. Une rangée, puis un mur — chapitre 6 : Dessiner. · gameboy3 — écrire une cartouche Game Boy en C++